

OS - Assignment 2

Q4.1)

1- Web Server Handling Multiple Client Requests

A single threaded web-server processes one request at a time, the subsequent request must wait, leading to latency.

A multi-threaded server would instead assign each client request to a separate thread. While one thread waits for a disk or network I/O, others can continue processing, increasing throughput and reducing response time.

2- Parallel Image Processing

Operations such as filtering, edge detection, or resizing on large images can be divided into independent regions (eg block of rows or columns). Each thread processes a different image segment simultaneously on separate CPU cores. This parallel execution reduces total processing time nearly proportionally to the number of cores.

3- Matrix Multiplication

When multiplying two large matrices, the computation of each row of the result matrix is independent of the others. A multi-threaded program can assign the computation of each row (or block of rows) to a separate thread, allowing all rows to be computed in parallel across multiple CPU cores. This reduces execution time proportionally to the no. of available cores.

Q4.2) Amdahl Law:- $Speed \leq \frac{1}{S + \left(\frac{1-S}{N}\right)}$

where S = serial portion so $1 - 0.6 = 0.4$

$(1-S)$ = parallel portion = 0.6

N = no. of process core

i) $N = 2$ core

$$Speedup \leq \frac{1}{0.4 + \frac{0.6}{2}}$$

$$Speed \leq 1.42$$

ii) $N = 4$ cores

$$Speedup \leq \frac{1}{0.4 + \frac{0.6}{4}}$$

$$Speed \leq 1.82$$

Q4.3) It exhibits task parallelism, reason being that task parallelism decomposes a problem into different, possibly unrelated tasks executed concurrently.

In a multi-threaded web server, each thread performs a similar but independent task - handling an entire HTTP request (listening, parsing, fetching files, sending response).

Q4.4) The two differences between user-level threads and Kernel-level threads is that:

- The user level threads are entirely managed by a user-space thread library (eg; POSIX Pthreads) and the kernel is unaware of them.

Whereas Kernel level threads are managed directly by the operating system kernel.

- Next is the blocking behaviour. If one user-level thread makes a blocking system call, the entire process blocks (since the kernel sees only one kernel thread).
- Meanwhile if one kernel-level thread blocks, other threads in the same process can continue executing.

User-level threads are better for applications with many short-live threads or frequent context switching as they avoid kernel overhead. However they are unsuitable for I/O-intensive tasks unless using a non-blocking I/O.

Kernel level thread are better for applications requiring true concurrency across multiple CPUs (parallel computation) or those making heavy use of blocking system calls as the kernel can schedule other threads while one blocks.

Q4.5) When the kernel context-switch between two kernel-level threads, it performs the following actions:

- 1- Save the content of current thread into its thread control block (TCB)
 - Program Counter(PC), CPU registers, Stack pointer (user and kernel stacks), processor status flags.
- 2- Update process and thread scheduling information.
 - Mark current thread as waiting / runnable depending on a reason for switch (eg:- time-slice expired, I/O wait).
 - Select next thread to run via scheduler, from the ready queue.

3- Restore the state of the new thread

The kernel loads the saved CPU register context from the incoming thread's TCB - (restoring its PC, SP and all registers) so that the CPU resumes execution exactly where that thread previously stopped.

4- Resume Execution

Control is transferred to the restored program counter of the newly scheduled thread, and execution continues.

Q4.6) When a Thread is created within an existing process, it requires only a minimal set of resources:

- TCB to store thread specific state
- PC to track where the thread is executing
- Register set - a private copy of CPU registers.
- Stack - a private stack for local variables and function call management.

The new thread shares with all other threads in the same

process: the code segment, data segment, heap memory, open files, and signals.

1st

When we create a new process (eg; via `fork()`) requires significantly more resources.

- A completely new PCB
- A separate memory address space (code, data, heap, stack all duplicate).
- Duplicated file descriptors and I/O resources.
- It's own set of signal handlers and OS accounting information.

The main difference is that thread creation is much cheaper (in time and memory) than process creation because threads share the address^{space} and resources of their parent process.

Process creation requires the OS to allocate and initialize an entirely independent memory space and PCB, making it far more resource intensive.

Hence why threads are often called lightweight processes.

Q4.7) Yes it is necessary to bind a real-time thread to a lightweight process.

In many-to-many model, user level threads are multiplexed over a pool of LWP's and the thread library decides which user thread runs on which LWP at any given time. This introduces scheduling non determinism (a user thread may not get immediate access to an LWP when it becomes runnable).

Real-time threads have strict timing and latency requirements. If a real time thread is not permanently bound to an LWP, it may have

to wait for an LWP to become available, causing unpredictable and potentially unacceptable delays that violate real-time constraints. By permanently binding a real-time thread to a dedicated LWP, the OS guarantees that whenever the real-time thread is ready to run, its LWP is immediately available to be scheduled onto a CPU by the kernel.

Q4.8) Trivial Arithmetic on Small Data Sets:

like adding two small arrays of 100 integers. The cost of creating, synchronizing and joining threads far exceeds the computation time. A single threaded solution completes faster due to lower overhead.

2- Highly Sequential Algorithms:-

Calculating the fibonacci sequence iteratively or computing a checksum that depends entirely on the previous step. These tasks can't be parallelized effectively. Multi-threading adds only contention for CPU resources without any true concurrency, degrading performance due to context switching overhead.

Q4.9) On a single processor system only one thread runs at a time. However multi threading with multiple kernel threads can still provide better performance in the following circumstances.

- When thread perform blocking I/O operations.

If one kernel thread blocks while waiting for I/O, the kernel can context-switch to another ready kernel thread that can continue executing useful CPU work. In a single threaded solution, the entire program stalls during every I/O wait.

Therefore on a single processor system, multithreading improves performance specifically for I/O bound applications, by overlapping computation and I/O.

For purely CPU bound tasks with no blocking, multithreading on a single processor provides no performance benefit and may slightly hurt performance due to context-switching overhead.

Q4.10) Component

Shared or Private

a. Register Value

Private (each thread has its own register set and program counter)

b. Heap Memory

Shared - all threads in a process share the same heap.

c. Global Variables

Shared - global/static data is part of the process data segment, shared by all threads.

d. Stack Memory

Private - each thread has its own stack for local variable and function calls.

Q4.11) No (not if they are purely user-level threads mapped under a ~~many~~^{one many} to one model) ~~then the system will have had been~~

Reason for No: In the many to one model all user level threads map to a single kernel thread. The kernel sees only one thread for the entire process, so it can only schedule that process on one CPU at a time. Even on a multiprocessor system, the additional CPU's can't be utilized because the kernel has no knowledge of the multiple user-level thread inside the process.

Hence a collection of pure user level threads will perform the

same on a multiprocessor as on a single processor. To exploit a multiprocessor, threads must be visible to kernel (i.e. many to many or one to one model).

Q4.12) No, the same benefits would not have been achieved with per-tab threads

Chrome opens each tab in a separate process deliberately for these reasons

1- Fault isolation:- If a tab crashes (due to a malicious script etc), only that process dies. Since process have completely separate memory spaces, a crash in one tab can't corrupt the memory of other. (whereas in thread due to shared address space it would have brought the whole browser down)

2- Security Isolation: Each process runs in it's own sandbox memory space making it much harder for a malicious webpage to access data from another tab. (whereas thread won't give this security)

Hence a thread tab design would be lighter weight and use fewer resources but it would sacrifice the crash isolation and security guarantees that are the entire motivation for chrome's multi process architecture.

Q4.13) Yes absolutely. For concurrency means that multiple tasks are in progress at the same time (they are being ~~context switched~~ managed and interleaved but not necessarily executing simultaneously).

On a single processor system, a multi threaded program is concurrent - the OS rapidly context-switches between threads giving the illusion of simultaneous progress. However, at any given clock cycle, only one thread is actually executing on the CPU. There is concurrency without parallelism.

While in Parallelism multiple tasks are literally executing at the same instant on multiple processing cores.

Q4.14)

$$i a) \text{ Speed} = \frac{1}{0.6 + \frac{0.4}{8}} = 1.54$$

$$b) \text{ Speed} = \frac{1}{0.6 + \frac{0.4}{16}} = 1.60$$

$$ii a) \text{ Speed} = \frac{1}{0.33 + \frac{0.67}{2}} = 1.50$$

$$b) \text{ Speed} = \frac{1}{0.33 + \frac{0.67}{4}} = 2.01$$

$$iii a) \text{ Speed} = \frac{1}{0.1 + \frac{0.9}{4}} = 3.08$$

$$b) \text{ Speed} = \frac{1}{0.1 + \frac{0.9}{8}} = 4.71$$

Q4.15)

i) Data Parallelism - same operation applied to different elements of data simultaneously

ii) Data Parallelism

iii) Task parallelism

iv) Data Parallelism

v) Both hybrid - GCD abstracts both paradigms: it can dispatch identical tasks on data partitions or ~~diff~~ (data parallelism) or different functions (task parallelism) using queues.

Q4.16) We will create one thread for input and one thread for output (or 1 thread to handle both sequentially)

Reason being that input occurs only at start-up (opening one file) and output occurs only at termination (writing one file). These two operations don't overlap with computation cause input finishes before CPU-bound work begins. Output starts after all computation ends. Then the ^{two} operations are sequential in nature. For a single file can't be read or written faster by multiple threads due to physical disk head constraints and file system locking.

Hence creating multiple I/O threads would add unnecessary overhead without any performance gain. In fact, concurrent writes to a single file would require complex locking and could corrupt data. Thus, one thread suffices.

ii) 4 threads (equal to the number of available processing cores).

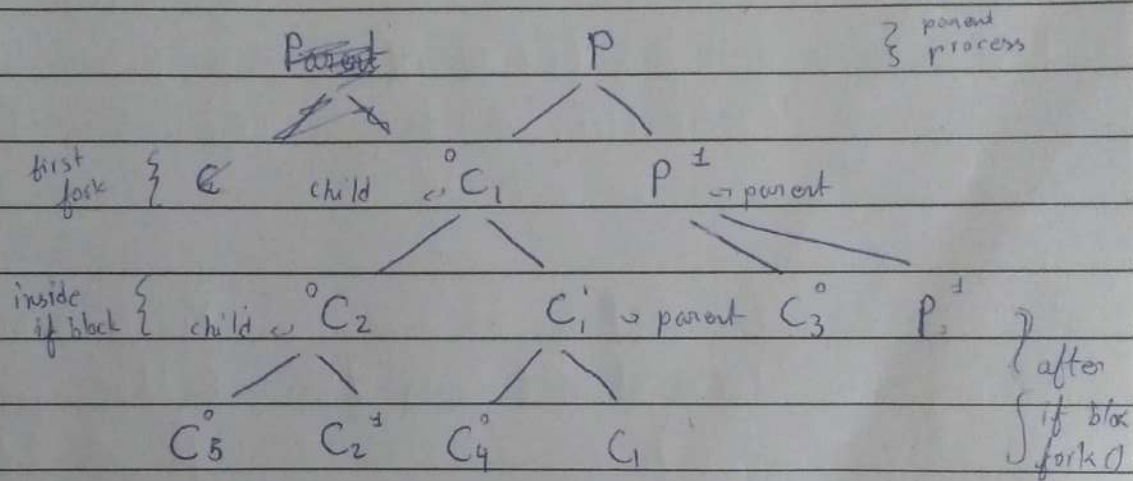
Reason being that the application is entirely CPU-bound between start-up and termination, and the system has 4 cores with a one to one

threading model, the optimal strategy is to create exactly one thread per core. This ensures all 4 cores remain utilized with no idle time. ☹

Creating more than 4 threads would cause unnecessary context switching without any additional parallelism, since only 4 threads can execute at the same time.

Creating fewer would leave cores idle and underutilize the hardware.

Q4.17)



Total process created : 5 (C_1, C_2, C_3, C_4, C_5) excluding original parent.

If we include parent there would be 6 procs

ii) Only 1 unique thread is created by C_1 process

Q4.18)

Q 4.18)

Aspect	Linux Approach	Windows Approach
- Fundamental Entity	A single structure - "task_struct" represents both processes and threads	Separate structures: EPROCESS (process) and ETHREAD (thread).
- Creation Mechanism	clone() system call with flags (CLONE_VM, CLONE_FILES, etc) to share resources.	CreateProcesses() creates a process with one thread. CreateThread() adds threads.
- Resource Sharing	Explicit: Flags determine what is shared (memory, files, signals). Implicit: Default flags give traditional process behavior.	Process owns resources (address space, handles). Threads belong to exactly one process and share its resources.
- Kernel data structure relationship	No separate process/thread hierarchy; each task has its own task_struct; sharing is via pointers to shared resource structures	EPROCESS contains a list of ETHREAD structures pointing to threads within that process.
- Context Switching	Same mechanism for all tasks (lightweight)	Switching between threads of same process is faster than switching between processes.

- | | | |
|-----------------|--|---|
| - Overhead | Uniform overhead. Flexibility comes from flag settings | Process creation heavier than thread creation. |
| - Advantages | Simpler kernel design; extreme flexibility
no artificial distinction | Clear conceptual separation
natural protection boundaries
easier for programmers to understand. |
| - Disadvantages | Less intuitive for programmer accustomed to classic process/thread distinction | More complex kernel code, less flexible can't partially share resources. |

Summary:

Linux uses a unified model where any task can share resources with any other task via `clone()` flags, blurring the process/thread line.

Windows uses a hierarchical model where a process is a container for one or more threads, with clear ownership and isolation boundaries.

Q4.19) Line C output $\Rightarrow$ CHILD: value = 5

as via `fork` the value = 0 is copied into child's own address space, and then the new thread created ~~resets~~ sets value = 5 within the child's address space. The `pthread_join()` ensures the child's main thread waits for the runner to finish before printing. Hence when Line C executes, value = 5 is in child.

Line P output - PARENT: value = 0

the wait(NULL) makes the parent to wait until child exits as the runner thread modified value in the child's own memory space the parent's copy of value remains untouched.
Hence 0 as output.

Q4.22) #include <pthread.h>

#include <stdio.h>

#include <stdlib.h>

double avg; } global
int min, int max;

typedef struct {

int *numbers;

int count;

~~double result;~~

} ThreadData;

void *calc_avg (void *param) {

ThreadData *data = (ThreadData *) param

double sum = 0.0;

for (int i=0; i < data->count; i++) {

sum += data->numbers[i]

}

~~data->result~~^{avg} = sum / data->count;

pthread_exit(0);

}


```

void * calc_min(void * param) {
    ThreadData * data = (ThreadData *) param;
    int min_val = data->numbers[0];
    for (int i = 1; i < data->count; i++) {
        if (data->numbers[i] < min_val)
            min_val = data->numbers[i];
    }
    data->result min = min_val;
    pthread_exit(0);
}

```

```

void * calc_max(void * param) {
    ThreadData * data = (ThreadData *) param;
    max max = data->numbers[0];
    for (int i = 1; i < data->count; i++) {
        if (data->number[i] > max) {
            max = data->numbers[i];
        }
    }
    pthread_exit(0);
}

```

```

int main(int argc, char * argv[]) {
    if (argc < 2) {
        printf("Insufficient input");
        return 1;
    }
    int count = argc - 1;

```

```
int * numbers = malloc (count * sizeof (int));  
for (int i = 0; i < count; i++)  
    numbers[i] = atoi (argv[i+1]);
```

```
ThreadData data = { numbers, count };
```

```
pthread_t tid1, tid2, tid3;
```

```
pthread_attr_t attr;
```

```
pthread_attr_init (&attr);
```

```
pthread_create (&tid1, &attr, calc_avg, &data);
```

```
pthread_create (&tid2, &attr, calc_max, &data);
```

```
pthread_create (&tid3, &attr, calc_min, &data);
```

```
pthread_join (tid1, NULL);
```

```
pthread_join (tid2, NULL);
```

```
pthread_join (tid3, NULL);
```

```
printf ("Avg: %.0f\n", avg);
```

```
printf ("Min: %d\n", min);
```

```
printf ("Max: %d\n", max);
```

```
free (numbers);
```

```
return 0;
```

```
}
```


Q 4.23]

#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

void * print_primes(void * param) {

int limit = *(int *) param;

printf("Prime numbers up to %d : \n", limit);

for (int num = 2; num <= limit; num++) {

int is_prime = 1;

for (int i = 2; i * i <= num; i++) {

if (num % i == 0) {

is_prime = 0;

break;

}

}

if (is_prime) {

printf("%d", num);

}

}

printf("\n");

pthread_exit(0);

}

int main(int argc, char * argv[]) {

if (argc != 2) {

printf("Usage : %s <number> \n", argv[0]);

return 1;

}

```
int limit = atoi(argv[1]);  
if (limit < 2) {  
    printf("Please enter a number >= 2.\n");  
    return 1;  
}  
  
pthread_t tid;  
pthread_attr_t attr;  
pthread_attr_init(&attr);  
pthread_create(&tid, &attr, print_primes, &limit);  
pthread_join(tid, NULL);  
return 0;
```



```
Q2) #include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <stdbool.h>
#include <pthread.h>
#include <math.h>
```

```
// tunable
```

```
#define N 1000
#define T_HOT 35.0f
#define T_Cold -10.0f
#define NUM_THREADS 5
```

```
// Satellite Data Types
```

```
typedef struct {
    float data[N][N];
    int satellite-id;
```

```
} Satellite Matrix;
```

```
typedef struct {
    float global_mat[N][N];
    int overlap-count[N][N];
    pthread_mutex_t row-mutex[N];
```

```
} Climate;
```

```
typedef struct {
```

```
Climate *mainmap;
SatelliteMatrix satellitemap;
} Satellite Worker Args;
```

Climate main-map, // Global variable

```
void initClimate (Climate *c) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            c->global-mat[i][j] = 0.0f;
            c->overlap-count[i][j] = 0;
        }
        pthread_mutex_init(&c->row-mutex[i], NULL);
    }
}
```

```
void generateSatelliteMatrix (SatelliteMatrix *res, int id) {
    res->satellite-id = id;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            res->data[i][j] = ((i+j) % 10 == 0) ? NAN : (float)(rand() % 70 - 20);
}
```

```
void interpolateNAN (float mat[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (!isnan(mat[i][j])) continue;
            float sum = 0.0f;

```



```
int cnt = 0;
```

```
if (i > 0 && !isnan(mat[i-1][j])) { sum += mat[i-1][j]; cnt++; }
```

```
if (i < N-1 && !isnan(mat[i+1][j])) { sum += mat[i+1][j]; cnt++; }
```

```
if (j > 0 && !isnan(mat[i][j-1])) { sum += mat[i][j-1]; cnt++; }
```

```
if (j < N-1 && !isnan(mat[i][j+1])) { sum += mat[i][j+1]; cnt++; }
```

```
mat[i][j] = (cnt > 0) ? sum / cnt : 20.0f;
```

```
}
```

```
}
```

```
}
```

```
void *satelliteWorker(void *args) {
```

```
    SatelliteWorkerArgs *s-args = (SatelliteWorkerArgs *)args;
```

```
    SatelliteMatrix *s-data = &s-args->satellite_map;
```

```
    Climate *c-map = s-args->mainmap;
```

```
    interpolateNaN(s-data->data);
```

```
    for (int i = 0; i < N; i++) {
```

```
        pthread_mutex_lock(&c-map->row-mutex[i]);
```

```
        for (int j = 0; j < N; j++) {
```

```
            c-map->global-mat[i][j] += s-data->data[i][j];
```

```
            c-map->overlap-count[i][j] += 1;
```

```
        }
```

```
        pthread_mutex_unlock(&c-map->row-mutex[i]);
```

```
    }
```

```
    printf("[satellite] Satellite %d processing complete.\n", s-data->satellite-id);
```

```
    return NULL;
```

```
}
```

// tile based Statistics

#define TILE-SIZE 100

#define NUM-TILES-X (N/TILE-SIZE)

#define TOTAL-TILES (NUM-TILES-X * NUM-TILES-X)

static int tile-index = 0;

static pthread_mutex_t tile-lock = PTHREAD_MUTEX_INITIALIZER;

typedef struct {

float global-max, global-min, global-sum, global-sum-sq;

int anomaly-count

pthread_mutex_t lock;

} GlobalStats;

GlobalStats g-stats = {

• global-max = -INFINITY;

• global-min = INFINITY;

• global-sum = 0.0f;

• global-sum-sq = 0.0f;

• anomaly-count = 0;

• lock = PTHREAD_MUTEX_INITIALIZER

};

typedef struct {

float (*matrix)[N];

GlobalStats *stats;

} TileWorkers Args;


```

static int getNextTile(void) {
    pthread_mutex_lock(&tile_lock);
    int idx = (tile_index < TOTAL_TILES) ? tile_index++ : -1;
    pthread_mutex_unlock(&tile_lock);
    return idx;
}

```

```

void *tileWorker(void *arg) {
    TileWorkerArgs *args = (TileWorkerArgs *)arg;
    float (*mat)[N] = args->matrix;
    GlobalStats *stats = args->stats;

```

```

    while(1) {
        int tile = getNextTile();
        if (tile == -1) break;

        int rowstart = (tile / NUM_TILES_X) * TILE_SIZE;
        int colstart = (tile % NUM_TILES_X) * TILE_SIZE;
        float local_min = INFINITY, local_max = -INFINITY;
        double local_sum = 0.0, local_sum_sq = 0.0;
        int cell_count = 0, local_anomalies = 0;

```

```

        for (int i = rowstart; i < rowstart + TILE_SIZE; i++) {
            for (int j = colstart; j < colstart + TILE_SIZE; j++) {
                float val = mat[i][j];
                if (val < local_min) { local_min = val; }
                if (val > local_max) { local_max = val; }
                local_sum += val;

```



```

    local-sum-sq += (double) val * val;
    cell-count++;
}
}

float local-mean = (float)(local-sum / cell-count);
float local-variance = (float)(local-sum-sq / cell-count) - local-mean * local-mean;
float local-std = sqrtf(local-variance);

for (int i = rowstart; i < rowstart + TILE-SIZE; i++) {
    for (int j = colstart; j < colstart + TILE-SIZE; j++) {
        if (fabsf(mat[i][j] - local-mean) > 2.0 * local-std)
            { local-anomalies++; }
    }
}

pthread_mutex_lock(&stats->lock);
if (local-min < stats->global-min) { stats->global-min = local-min; }
if (local-max > stats->global-max) { stats->global-max = local-max; }
stats->global-sum += (float) local-sum;
stats->global-sum-sq += (float) local-sum-sq;
stats->anomaly-count += local-anomalies;
pthread_mutex_unlock(&stats->lock);
}

return NULL;
}

```

// task parallel phase global.

bool is-hotspot[N][N] = {false};

bool is-coldspot[N][N] = {false};


```
bool near-both [N][N] = {false};
```

```
int rows-processed-A = 0;
```

```
pthread_mutex_t progress-lock = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_cond_t progress-cond = PTHREAD_COND_INITIALIZER;
```

```
int hotspot-count = 0, coldspot-count = 0;
```

```
pthread_mutex_t count-lock = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t flag-mutex = PTHREAD_MUTEX_INITIALIZER;
```

```
float normalized-mat [N][N]
```

```
float risk-score [N][N];
```

```
static pthread_barrier_t barrier-abc; // A, B, C signal primary work done
static pthread_barrier_t barrier-ick; // C → Risk hand off.
```

```
void mark-near-both (int i, int j, bool is-hot) {
```

```
    for (int di = -2; di <= 2; di++) {
```

```
        for (int dj = -2; dj <= 2; dj++) {
```

```
            if (abs(di) + abs(dj) <= 2) { continue; }
```

```
            int ni = i + di;
```

```
            int nj = j + dj;
```

```
            if (ni < 0 || ni >= N || nj < 0 || nj >= N) { continue; }
```

```
            if (is-hot && is-coldspot[ni][nj]) {
```

```
                near-both[i][j] = true
```

```
                near-both[ni][nj] = true;
```

```
            }
```

```
            if (!is-hot && is-hotspot[ni][nj]) {
```



```

    near-both[i][j] = true;
    near-both[ni][nj] = true;

```

```

}

```

```

}

```

```

}

```

```

}

```

// TASK A: HOTSPOT DETECTION

```

void * taskA-hotspots (void *arg) {
    (void) arg;

```

```

    for (int i=0; i < N; i++) {

```

```

        for (int j=0; j < N; j++) {

```

```

            if (main-map-global-mat[i][j] > T-Hot) {

```

```

                pthread-mutex-lock(&flag-mutex);

```

```

                is-hotspot[i][j] = true;

```

```

                mark-near-both(i, j, true);

```

```

                pthread-mutex-unlock(&flag-mutex);

```

```

                pthread-mutex-lock(&count-lock);

```

```

                hotspot-count++;

```

```

                pthread-mutex-unlock(&count-lock);

```

```

            }

```

```

        }

```

```

// Signal Task C that row i is ready

```

```

pthread-mutex-lock(&progress-lock);

```

```

rows-processed-A = i+1;

```

```

pthread-cond-broadcast(&progress-cond);

```

```

pthread-mutex-unlock(&progress-lock);

```



```

}
printf("[taskA] Hotspot detection done - %d hotspots found.\n",
hotspot-count);
pthread_barrier_wait(&barrier-abc);
return NULL;
}

```

```

void *taskB-coldspots(void *arg) {
    (void) arg;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (main-map.global-mat[i][j]
                pthread_mutex_lock(&flag-mutex);
                is-coldspot[i][j] = true;
                mark-near-both(i, j, false);
                pthread_mutex_unlock(&flag-mutex);

                pthread_mutex_lock(&count-lock);
                coldspot-count++;
                pthread_mutex_unlock(&count-lock);
            }
        }
        pthread_mutex_lock(&progress-lock);
        rows-processed-B = i + 1;
        pthread_cond_broadcast(&progress-cond);
        pthread_mutex_unlock(&progress-lock);
    }
}

```

```

printf("[taskB] Coldspot detection done - %d coldspots found.\n",

```

```

coldspot-count);
pthread_barrier_wait(&barrier-abc);
return NULL;
}

```

// Task C: normalize

```

void * taskC_normalize(void *arg) {

```

```

    (void) arg;

```

```

    float global-min = g-stats.global-min;

```

```

    float global-max = g-stats.global-max;

```

```

    float range = global-max - global-min;

```

```

    if (range == 0.f) { range = 1.f; }

```

```

    int processed-rows = 0;

```

```

    while (processed-rows < N) {

```

```

        pthread_mutex_lock(&progress-lock);

```

```

        while (1) {

```

```

            int ra = rows-processed-A, rb = rows-processed-B;

```

```

            int ready = (ra < rb) ? ra : rb;

```

```

            if (ready > processed-rows) { break; } // new row available

```

```

            if (ra == N && rb == N) { break; } // both full done

```

```

            pthread_cond_wait(&progress-cond, &progress-lock);
        }

```

```

    int ra = rows-processed-A, rb = rows-processed-B;

```

```

    pthread_mutex_unlock(&progress-lock);

```

// Determine now how many rows are safe to normalise


```

int ready-rows = (ra < rb) ? ra : rb;
if (ra == N && rb == N) { ready-rows = N; }
if (ready-rows < processed-rows) { ready-rows = processed-rows; }

```

```

for (int i = processed-rows; i < ready-rows; i++) {
    for (int j = 0; j < N; j++) {
        float val = main-map.global-mat[i][j];
        normalised-mat[i][j] = (val - global-min) / range;
    }
}

```

```

processed-rows = ready-rows
}

```

```

pthread_barrier_wait (& barrier-abc); // wait for A, B to complete

```

```

for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        if (near-both[i][j])

```

```

            normalized-mat[i][j] += 0.5;

```

```

printf("[task C] Normalisation (+ near-both adjustment) done\n");

```

```

pthread_barrier_wait (& barrier-risk);

```

```

return NULL;
}

```

// Risk Score Thread.

```

void *riskScoreWorker (void *arg) {

```

```

    (void) arg;

```

```

    pthread_barrier_wait (& barrier-risk);

```

```

    printf("[risk Score] Risk score thread started.\n");

```

```
static float prox-hot[N][N], prox-cold[N][N];
memset(prox-hot, 0, sizeof(prox-hot));
memset(prox-cold, 0, sizeof(prox-cold));
```

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        if (is-hotspot[i][j] || is-coldspot[i][j]) {
            for (int di = -2; di <= 2; di++) {
                for (int dj = -2; dj <= 2; dj++) {
                    int dist = abs(di) + abs(dj);
                    if (dist > 2) continue;
                    int ni = i + di;
                    int nj = j + dj;
                    if (ni < 0 || ni >= N || nj < 0 || nj >= N) {
                        continue;
                    }
                    float p = 1.0f / (float)(dist + 1);
                    if (is-hotspot[i][j] && p > prox-hot[ni][nj])
                        prox-hot[ni][nj] = p;
                    if (is-coldspot[i][j] && p > prox-cold[ni][nj])
                        prox-cold[ni][nj] = p;
                }
            }
        }
    }
}
```


//min heap for top 10

```
typedef struct {
```

```
    int i, j;
```

```
    float score;
```

```
} RiskEntry;
```

```
RiskEntry heap[10];
```

```
int heap-size = 0;
```

```
for (int i = 0; i < N; i++) {
```

```
    for (int j = 0; j < N; j++) {
```

```
        float score = normalized_mat[i][j] * prox-hot[i][j] / (prox-cold[i][j] + 1.0f);
```

```
        risk-score[i][j] = score;
```

```
    if (heap-size < 10) {
```

```
        heap[heap-size++] = (RiskEntry){i, j, score};
```

```
        int idx = heap-size - 1;
```

```
        while (idx > 0) {
```

```
            int parent = (idx - 1) / 2;
```

```
            if (heap[idx].score < heap[parent].score) {
```

```
                RiskEntry tmp = heap[idx];
```

```
                heap[idx] = heap[parent];
```

```
                heap[parent] = tmp;
```

```
                idx = parent;
```

```
            } else {
```

```
                break;
```

```
        }
```

```
    }
```

```

} else if (score > heap[0].score) {
    heap[0] = (RiskEntry) {i, j, score};
    int idx = 0;
    while (1) {
        int L = 2 * idx + 1, r = 2 * idx + 2, sm = idx;
        if (L < heap-size && heap[L].score < heap[sm].score)
            sm = L;
        if (r < heap-size && heap[r].score < heap[sm].score)
            sm = r;
        if (sm == idx) break;
        RiskEntry tmp = heap[idx]; heap[idx]
        heap[idx] = heap[sm];
        heap[sm] = tmp;
        idx = sm;
    }
}
}
}

```

```

}
// Sort top 10 descending
for (int i = 0; i < heap-size - 1; i++) {
    int mx = i;
    for (int k = i + 1; k < heap-size; k++)
        if (heap[k].score > heap[mx].score)
            mx = k;
    if (mx != i) {
        RiskEntry tmp = heap[i];
        heap[i] = heap[mx];
        heap[mx] = tmp;
    }
}

```



```

    }
    printf("\n Top 10 Risk Cells : \n");
    for (int k=0; k < heap-size; k++)
        printf(" Rank %2d: (%.4d, %.4d) score=%.6f \n",
            k+1, heap[k].i, heap[k].j, heap[k].score);

    return NULL;
}

```

```

int main (void) {
    srand (Consigned)time (NULL));
    initClimate (&main-map);

```

// Phase 1: Satellite merge

```

pthread_t threads [NUM-THREADS];

```

```

SatelliteWorkerArgs * sat_args = malloc (sizeof (SatelliteWorkerArgs) *
                                            NUM-THREADS);

```

```

if (!sat_args) { perror ("malloc"); return 1; }

```

```

for (int i=0; i < NUM-THREADS; i++) {

```

```

    sat_args[i].mainmap = &main-map;

```

```

    generateSatelliteMatrix (&sat_args[i].satellitemap, i+1);

```

```

    pthread_create (&threads[i], NULL, satelliteWorker, &sat_args[i]);

```

```

}

```

```

for (int i=0; i < NUM-THREADS; i++) {

```

```

    pthread_join (threads[i], NULL);

```

```

}

```

// avg overlapping cells.


```
for (int i=0; i<N; i++)
```

```
for (int j=0; j<N; j++)
```

```
main-map.global-mat[i][j] = (main-map.overlap-count  
[i][j]>0) ? main-map.global-mat[i][j]/main-map  
* overlap-count[i][j] : 20.f;
```

```
// destroy per row mutexes now that merge is complete  
for (int i=0; i<N; i++)
```

```
pthread_mutex_destroy(&main-map.row-mutex[i]);
```

```
printf("Global Matrix Merged and Averaged. \n");
```

```
// Phase 2: Tile Statistics
```

```
int num-tile-threads = 8;
```

```
pthread_t tile-threads[num-tile-threads];
```

```
Tile Worker Args tile-args = {  
    * matrix = main-map.global-mat,  
    * stats = &g-stats  
};
```

```
for (int i=0; i<num-tile-threads; i++)
```

```
pthread_create(&tile-threads[i], NULL, tile Worker, &tile-args);
```

```
for (int i=0; i<num-tile-threads; i++)
```

```
pthread_join(tile-threads[i], NULL);
```

```
float global-mean = g-stats.global-sum / (float)(N * N);
```

```
float global-variance = g-stats.global-sum-sq / (float)(N * N) - global-mean  
* global-mean;
```



```

printf("Global Statistics: \n");
printf("Max: %.2f \n", g-stats.global-max);
printf("Min: %.2f \n", g-stats.global-min);
printf("Variance: %.2f \n", global-variance);
printf("Mean: %.2f \n", global-mean);
printf("Anomalies (>2 sigma): %d \n", g-stats.anomaly-count);

```

// Phase 3: task parallel (A, B, C, Risk)

```

pthread_barrier_init(&barrier-abc, NULL, 3);
pthread_barrier_init(&barrier-risk, NULL, 2);

```

```

pthread_t threadA, threadB, threadC, threadRisk;
pthread_create(&threadA, NULL, taskA-hotspots, NULL);
pthread_create(&threadB, NULL, taskB-coldspots, NULL);
pthread_create(&threadC, NULL, taskC-normalize, NULL);
pthread_create(&threadRisk, NULL, riskScoreWorker, NULL);

```

```

pthread_join(threadA, NULL);
pthread_join(threadB, NULL);
pthread_join(threadC, NULL);
pthread_join(threadRisk, NULL);

```

```

int near-both-count = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++)
        if (near-both[i][j]) {near-both-count++;}
}

```

```

printf("\n Near both cells (entire matrix): %d \n", near-both-count);

```



```
pthread_barrier_destroy(&barrier_alc);
pthread_barrier_destroy(&barrier_risk);
```

// final Output

```
printf("\n Hotspots: %d\n", hotspot-count);
printf("\n Coldspots: %d\n", coldspot-count);
```

```
printf("\n Sample 5x5 normalized matrix (top-left): \n");
for (int i=0; i<5; i++) {
    for (int j=0; j<5; j++)
        printf("%.7f", normalized_mat[i][j]);
    printf("\n");
}
```

```
printf("\n Sample 5x5 weighted-adjustment flags (top-left): \n");
printf("(ADJ= near_both * 0.5 applied; --- = normal) \n");
for (int i=0; i<5; i++) {
    for (int j=0; j<5; j++)
        printf("%.5", near_both[i][j] ? "ADJ" : "---");
    printf("\n");
}
```

```
pthread_mutex_destroy(&tile_lock);
pthread_mutex_destroy(&g-stats_lock);
pthread_mutex_destroy(&progress_lock);
pthread_mutex_destroy(&count_lock);
pthread_mutex_destroy(&flag_mutex);
free(sat_args);
return 0;
```



```
Q3) #include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
```

```
#define CRITICAL 0
#define SERIOUS 1
#define NORMAL 2
#define JUNIOR 0
#define SENIOR 1
#define MAX-WAIT 30
#define SERIOUS-THRESHOLD 5
#define NORMAL-RUN-LIMIT 3
#define MAX-PATIENTS 20
```

```
typedef struct {
    int id, type, treated, assigned-doctor;
    time_t arrival-time;
    pthread_cond_t done;
} Patient;
```

```
typedef struct {
    int id, is-senior, normal count;
} Doctor;
```

```
// Three simple Circular Queues
#define QMAX 30
```

```
static Patient * queue [3][QMAX];
static int qhead [3] = {0}, qtail [3] = {0}, qsize [3] = {0};
```

```
static void q-push (int lvl, Patient *p) {
    if (qsize[lvl] >= QMAX) {
        printf ("Error overflow");
        return;
    }
    queue [lvl][qtail [lvl]] = p;
    qtail [lvl] = (qtail [lvl] + 1) % QMAX;
    qsize [lvl]++;
}
```

```
static Patient *q-pop (int lvl) {
    if (qsize[lvl] == 0) return NULL;
    Patient *p = queue [lvl][qhead [lvl]];
    qhead [lvl] = (qhead [lvl] + 1) % QMAX;
    qsize [lvl]--;
    return p;
}
```

```
static int total-patients = MAX-PATIENTS;
static int completed-patients = 0;
static int simulation-done = 0;
```

```
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t any-patient = PTHREAD_COND_INITIALIZER;
```



```
static const char *pname (int t) {
    return t == CRITICAL ? "Critical" : t == SERIOUS ? "Serious" :
        "Normal";
}
```

```
static const char *dname (int s) {
    return s == SENIOR ? "Senior" : "Junior";
}
```

// Rule 3

```
static void check-threshold (void) {
    if (qsize[SERIOUS] >= SERIOUS-THRESHOLD) {
        Patient *p = q-pop(SERIOUS);
        p->type = CRITICAL;
        q-push(CRITICAL, p);
        printf("[PROMOTE] Patient %ld: Serious → Critical (queue  
threshold reached)\n", p->id);
    }
}
```

// Rule 5

```
static void check-timeouts (void) {
    time_t now = time(NULL);
    int n = qsize[SERIOUS];

    for (int i = 0; i < n; i++) {
        Patient *p = q-pop(SERIOUS);
        if (now - p->arrival-time >= MAX_WAIT) {
```

$p \rightarrow \text{type} = \text{CRITICAL};$

$q\text{-push}(\text{CRITICAL}, p);$

$\text{printf}("[\text{PROMOTE}] \text{Patient } \%d: \text{Serious} \rightarrow \text{Critical } (\%d: \text{timeout}) \backslash n", p \rightarrow \text{id}, \text{MAX_WAIT});$

$\}$ else $\{$

$q\text{-push}(\text{SERIOUS}, p);$

$\}$

$\}$

$n = q\text{size}[\text{NORMAL}];$

for (int $i = 0; i < n; i++$) $\{$

Patient $* p = q\text{-pop}(\text{NORMAL});$

if ($\text{now} - p \rightarrow \text{arrival-time} \geq \text{MAX_WAIT}$) $\{$

$p \rightarrow \text{type} = \text{CRITICAL};$

$q\text{-push}(\text{CRITICAL}, p);$

$\text{printf}("[\text{PROMOTE}] \text{Patient } \%d: \text{Normal} \rightarrow \text{Critical } (\%d: \text{timeout}) \backslash n", p \rightarrow \text{id}, \text{MAX_WAIT});$

$\}$ else $\{$

$q\text{-push}(\text{NORMAL}, p);$

$\}$

$\}$

$\}$

static void $* \text{watchdog-thread}(\text{void} * \text{arg}) \{$

(void) $\text{arg};$

while (1) $\{$

sleep(1);

$\text{pthread_mutex_lock}(\&\text{mtx});$

if (simulation-done) $\{$


```
pthread_mutex_unlock(&mtx);
break;
```

```
}
```

```
check-timeouts();
```

```
pthread_cond_broadcast(&any-patient);
```

```
pthread_mutex_unlock(&mtx);
```

```
}
```

```
return NULL;
```

```
}
```

// Select Patient

```
static Patient *select-patient(Doctor *dc){
```

```
if (dc->is-senior == JUNIOR && qsize[CRITICAL] > 0)
```

```
return NULL; // Junior can't treat critical, wait for senior
```

```
if (dc->is-senior == SENIOR){
```

```
if (qsize[CRITICAL] > 0) return q-pop(CRITICAL);
```

```
if (dc->normal-count >= NORMAL-RUN-LIMIT && qsize[SERIOUS] > 0){
```

```
return q-pop(SERIOUS);
```

```
}
```

```
if (qsize[SERIOUS] > 0) {return q-pop(SERIOUS);}
```

```
if (qsize[NORMAL] > 0) {return q-pop(NORMAL);}
```

```
} else {
```

```
if (dc->normal-count >= NORMAL
```



```

static Patient *select-patient (Doctor *doc) {
    if (doc->is-senior == JUNIOR && qsize [CRITICAL] > 0) return NULL;

    if (doc->is-senior == SENIOR && qsize [CRITICAL] > 0) {
        return q-pop (CRITICAL);
    }
    if (doc->normal-count >= NORMAL-RUN-LIMIT && qsize [SERIOUS] > 0)
        return q-pop (SERIOUS);
    if (qsize [SERIOUS] > 0) { return q-pop (SERIOUS); }
    if (qsize [NORMAL] > 0) { return q-pop (NORMAL); }
    return NULL;
}

```

// patient storage

```

static Patient all-patients [MAX-PATIENTS];

```

```

static void *patient-thread (void *arg) {
    Patient *p = (Patient *)arg;
    pthread-mutex-lock (& mtn);

```

```

    p->arrival-time = time (NULL);

```

```

    printf ("[ARRIVE] Patient %d arrived as %s\n", p->id, pname (p->type));

```

```

    q-push (p->type, p);

```

```

    check-threshold (); // Rule 3

```

```

    pthread-cond-broadcast (& any-patient);

```

```

    while (!p->treated)

```

```

        pthread-cond-wait (& p->done, & mtn);

```



```

printf("[TREATED] Patient %d (%s) by Doctor %d\n", p->id,
        p->name(p->type), p->assigned-doctor);
pthread_mutex_unlock(&mtx);
return NULL;
}

```

// Doctor thread

```

static void *doctor-thread(void *arg) {
    Doctor *doc = (Doctor *)arg;
    while (1) {
        pthread_mutex_lock(&mtx);
        Patient *p = select-patient(doc);

        while (p == NULL && completed-patients < total-patients) {
            pthread_cond_wait(&any-patient, &mtx);
            p = select-patient(doc);
        }

        if (completed-patients >= total-patients) {
            pthread_mutex_unlock(&mtx);
            break;
        }

        p->assigned-doctor = doc->id;
        printf("[START] Doctor %d (%s) -> Patient %d (%s)\n",
                doc->id, doc->is-senior, p->id, p->name(p->type));
        pthread_mutex_unlock(&mtx);
    }
}

```

sleep(2); // Simulate Treatment

```
pthread_mutex_lock (& mtn);
```

```
doc->normal-count = (p->type == NORMAL) ? doc->normal-count + 1  
: 0;
```

```
p->treated = 1;
```

```
completed-patients++;
```

```
pthread_cond_signal (& p->done);
```

```
pthread_cond_broadcast (& any-patient);
```

```
pthread_mutex_unlock (& mtn);
```

```
}
```

```
printf("Exit Doctor %d (%s) off duty.\n", doc->id, dname (  
doc->is-senior));
```

```
return NULL;
```

```
}
```

```
int main() {
```

```
Doctor doctors[4] = {
```

```
{1, SENIOR, 0}, {2, SENIOR, 0}, {3, JUNIOR, 0},  
{4, JUNIOR, 0}
```

```
};
```

```
int data[MAX-PATIENTS][2] = {
```

```
{1, NORMAL}, {2, SERIOUS}, {3, NORMAL}, {4, NORMAL},
```

```
{5, SERIOUS}, {6, SERIOUS}, {7, SERIOUS}, {8, SERIOUS},
```

```
{9, SERIOUS},
```

// Rule 3 of 5 continuous serious in queue, promote

```
{10, CRITICAL}, {11, NORMAL}, {12, NORMAL},
```

```
{13, NORMAL}, // Rule 4 3 consecutive normal
```

```
{14, SERIOUS}, {15, CRITICAL}, {16, NORMAL},
```

```
{17, SERIOUS}, {18, NORMAL}, {19, NORMAL}, {20, SERIOUS}
```

```
};
```



```

for (int i=0; i < MAX-PATIENTS; i++) {
    all-patients[i].id = data[i][0];
    all-patients[i].type = data[i][7];
    all-patients[i].treated = 0;
    all-patients[i].assigned-doctor = -1;
    all-patients[i].arrival-time = 0;
    pthread-cond-init (&all-patients[i].done, NULL);
}

```

```

printf("Hospital Emergency Department Simulation\n");

```

```

pthread_t dt[4], pt[MAX-PATIENTS], wt;
pthread-create (&wt, NULL, watchdog-thread, NULL);
for (int i=0; i < 4; i++)
    pthread-create (&dt[i], NULL, doctor-thread, &doctors[i]);
for (int i=0; i < MAX-PATIENTS; i++) {
    pthread-create (&pt[i], NULL, patient-thread, &all-patients[i]);
    usleep(400000); // arrival rate of patient
}

```

```

for (int i=0; i < MAX-PATIENTS; i++)
    pthread-join (pt[i], NULL);

```

```

// signal doctors they can exit
pthread-mutex-lock (&mtx);
pthread-cond-broadcast (&any-patient);
pthread-mutex-unlock (&mtx);

```

```
for (int i=0; i<4; i++)  
    pthread_join (dt[i], NULL);
```

```
// stop timing watchdog  
pthread_mutex_lock (&mtx);  
simulation_done = 1;  
pthread_mutex_unlock (&mtx);  
pthread_join (wt, NULL);
```

```
for (int i=0; i<MAX-PATIENTS; i++)  
    pthread_cond_destroy (&all-patients[i].done);
```

```
printf ("All %d patients treated successfully. \n", total-patients);
```

```
pthread_mutex_destroy (&mtx);  
pthread_cond_destroy (&any-patient);  
return 0;
```

```
}
```



```
Q4) #include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <string.h>
```

```
#define MAX-FLIGHTS 30
#define NUM-RUNWAYS 3
#define WAIT-THRESHOLD 5
#define TOTAL-FLIGHTS 15
```

```
#define EMERGENCY 100
#define FUEL-LOW 40
#define LANDING 20
#define TAKEOFF 10
#define CARGO-TAKEOFF 5
```

```
typedef struct {
    int id, priority, type;
    long arrival-time;
    char label[32];
} Flight;
```

```
static Flight heap[MAX-FLIGHTS];
static int heap-size = 0;
```

```
static void heap-swap(int a, int b) {
```

```

Flight tmp = heap[a]; h
heap[a] = heap[b];
heap[b] = tmp;

```

```

}

```

```

static void sift-up(int i) {

```

```

    while (i > 0) {

```

```

        int p = (i-1)/2;

```

```

        if (heap[p].priority >= heap[i].priority) break;

```

```

        heap-swap(p, i);

```

```

        i = p;

```

```

    }

```

```

}

```

```

static void sift-down(int i) {

```

```

    while (1) {

```

```

        int best = i, l = 2*i+1, r = 2*i+2;

```

```

        if (l < heap-size && heap[l].priority > heap[best].priority)
            best = l;

```

```

        if (r < heap-size && heap[r].priority > heap[best].priority)
            best = r;

```

```

        if (best == i) break;

```

```

        heap-swap(i, best);

```

```

        i = best;

```

```

    }

```

```

}

```



```

static void pq-push (Flight f) {
    if (heap-size >= MAX-FLIGHTS) {
        fprintf (stderr, "[WARN] Queue full - flight %d dropped", f.id);
        return;
    }
    heap[heap-size++] = f;
    sift-up (heap-size-1);
}

```

```

static int pq-pop (Flight * out) {
    if (heap-size == 0) return 0;
    long now = time (NULL);

```

```

    int heap-changed = 0;
    for (int i = 0; i < heap-size; i++) {
        long waited = now - heap[i].arrival-time;
        if (waited > WAIT-THRESHOLD) {
            int boost = (int)(waited - WAIT-THRESHOLD);
            int new-p = heap[i].priority + boost;
            if (new-p >= EMERGENCY) new-p = EMERGENCY - 1;
            if (new-p != heap[i].priority) {
                heap[i].priority = new-p;
                heap-changed = 1;
            }
        }
    }
}

```

```

// Rebuild heap to avoid mid-iteration corruption.
if (heap-changed) {

```

```

for (int i = heap-size / 2 - 1; i >= 0; i--)
    sift-down(i);
}

```

// Rule 3:

```

int land_idx = -1;
for (int i = 0; i < heap-size; i++) {
    if ((heap[i].type == LANDING || heap[i].type == FUEL-LOW) &&
        (now - heap[i].arrival-time > WAIT-THRESHOLD)) {
        if (land_idx == -1 || heap[i].priority > heap[land_idx].
            priority) {
            land_idx = i;
        }
    }
}
}

```

```

int pick = 0;
if (land_idx != -1) {
    int top-is-takeoff = (heap[0].type == TAKEOFF || heap[0].type
        == CARGO-TAKEOFF);
    if (top-is-takeoff || heap[land_idx].priority > heap[0].priority)
        pick = land_idx;
}

```

```

* out = heap[pick];
heap[pick] = heap[--heap-size];
sift-down(pick);
if (pick > 0) sift-up(pick);
return 1;
}

```


// flight waiting Time map :- Maps flight ID $\rightarrow$ arrival ^{time} Date: _____

#define MAX-ID(TOTAL-FLIGHTS + 1)

static long waiting-map[MAX-ID];

static void wmap-add (int id, long arrival) {

if (id > 0 && id < MAX-ID) waiting-map[id] = arrival;

}

static long wmap-get (int id) {

if (id > 0 && id < MAX-ID) {return waiting-map[id];}

return 0;

}

static void wmap-remove (int id) {

if (id > 0 && id < MAX-ID) waiting-map[id] = 0;

}

static int runway-busy[NUM-RUNWAYS] = {0};

static int runway-maint[NUM-RUNWAYS] = {0};

static pthread-mutex-t runway-lock [NUM-RUNWAYS];

static pthread-mutex-t queue-lock = PTHREAD-MUTEX-INITIALIZER;

static pthread-cond-t queue-cond = PTHREAD-COND-INITIALIZER;

static int flights-done = 0;

static int emergency-pending = 0;

static const char * type-name (int t) {

switch (t) {

case EMERGENCY: return "EMERGENCY";

case FUEL-LOW: return "FUEL-LOW";


```

case LANDING: return "LANDING";
case TAKEOFF: return "TAKEOFF";
case CARGO-TAKEOFF: return "CARGO";
default: return "UNKNOWN";
}
}

```

```

static void *flight-generator (void *arg) {
    (void) arg;
    int types[] = {LANDING, TAKEOFF, CARGO-TAKEOFF, FUEL-LOW,
                  EMERGENCY};
    int weights[] = {40, 30, 15, 10, 5};

```

```

    for (int i=0; i < TOTAL-FLIGHTS; i++) {
        int r = rand() % 100, cum = 0, t = 0;
        for (int k=0; k < 5; k++) {
            cum += weights[k];
            if (r < cum) { t = k; break; }
        }
    }

```

```

    Flight f;
    f.id = i+1;
    f.type = types[t];
    f.priority = types[t];
    f.arrival-time = time(NULL);
    snprintf(f.label, sizeof(f.label), "%s # %d", type-name(f.type),
             f.id);

```

```

pthread_mutex_lock(&queue-lock);

```



```

pg-push(f);
wmap-add(f.id, f.arrival-time);
printf("[GEN] %-20s priority = %.3d\n", f.label, f.priority);
pthread-cond-broadcast(&queue-cond);
pthread-mutex-unlock(&queue-lock);

```

```

usleep(400000); // 0.4 s between arrival.

```

```

}

```

```

return NULL;

```

```

}

```

```

static void *runway-controller(void *arg) {

```

```

    int rw = *(int *)arg;

```

```

    while(1) {

```

```

        pthread-mutex-lock(&queue-lock);

```

```

        while(1) {

```

```

            if (flights-done >= TOTAL-FLIGHTS && heap-size == 0) {

```

```

                pthread-mutex-unlock(&queue-lock);

```

```

                printf("[RWY %-2d] Controller exiting. \n", rw + 1);

```

```

                return NULL;

```

```

            }

```

```

            pthread-mutex-lock(&runway-lock[rw]);

```

```

            int maint = runway-main[rw];

```

```

            int busy = runway-busy[rw];

```

```

            pthread-mutex-unlock(&runway-lock[rw]);

```

```

            if (heap-size > 0 && !maint && !busy) break;

```

```

            pthread-cond-wait(&queue-cond, &queue-lock);

```

```

        }

```


Flight f;

if (!pq-pop(&f)) {

pthread_mutex_unlock(&queue-lock);

continue;

}

// Mark runway busy

pthread_mutex_lock(&runway-lock[rw]);

runway-busy[rw] = 1;

pthread_mutex_unlock(&runway-lock[rw]);

long arrival = utmap-get(f.id);

long waited = time(NULL) - arrival;

utmap-remove(f.id);

printf("[RWF %-2d] %-20s priority= %-3d waited %-1ds → ASSIGNED",
rw+1, f.label, f.priority, waited);

pthread_mutex_unlock(&queue-lock);

int op-time = (f.type == EMERGENCY) ? 4 : 2;

for (int s=0; s<op-time; s++) {

sleep(1);

pthread_mutex_lock(&queue-lock);

int ep = emergency-pending;

pthread_mutex_unlock(&queue-lock);

if (ep && f.type != EMERGENCY) {

printf("[RWV %-2d] %-20s → PREEMPTED (emergency
waiting) \n", rw+1, f.label);


```

pthread_mutex_lock (&queue_lock);
pq-push (f);
wmap-add (f.id, f.arrival-time);
pthread_cond_broadcast (&queue_cond);
pthread_mutex_unlock (&queue_lock);

```

```

pthread_mutex_lock (&runway_lock[rw]);
runway-busy[rw]=0;
pthread_mutex_unlock (&runway_lock[rw]);
goto next-flight;
}
}

```

// flight completed normally

```

pthread_mutex_lock (&queue_lock);
pthread_mutex_lock (&runway_lock[rw]);
runway-busy[rw]=0;
pthread_mutex_unlock (&runway_lock[rw]);

```

flights-done++;

```

if (f.type == EMERGENCY) {
    int more = 0;

```

```

    for (int i=0; i < heap-size; i++) {

```

```

        if (heap[i].type == EMERGENCY) {
            more = 1;

```

```

            break;

```

```

        }

```

```

    }

```

```

    if (!more) { emergency-pending = 0; }

```

```

}

```

```
printf("[RWY %-2d] %-20s → DONE (%d / %d) \n", fw+1, f-label,
        flights-done, TOTAL-FLIGHTS);
pthread_cond_broadcast(&queue-cond);
pthread_mutex_unlock(&queue-lock);
```

```
    next_flight++;
}
```

```
return NULL;
}
```

```
static void *emergency-monitor(void *arg) {
```

```
    (void) arg;
```

```
    while (1) {
```

```
        sleep(2);
```

```
        pthread_mutex_lock(&queue-lock);
```

```
        if (flights-done ≥ TOTAL-FLIGHTS && heap-size == 0) {
```

```
            pthread_mutex_unlock(&queue-lock);
```

```
            break;
```

```
        }
```

```
    int found-emergency = 0;
```

```
    for (int i=0; i < heap-size; i++) {
```

```
        if (heap[i].type == EMERGENCY) {
```

```
            found-emergency = 1;
```

```
            if (heap[i].priority < EMERGENCY) {
```

```
                heap[i].priority = EMERGENCY;
```

```
            }
```



```

    printf("EMRG ] %i ~20s detected - priority set to %d,  

    preemption flag SET\n", heap[i].label, heap[i].priority);
}
}

```

```

if (found-emergency) {
    for (int r=heap-size/2-1; r >= 0; r--)
        sift-down(r);
    emergency-pending = 1;
    pthread-cond-broadcast(&queue-cond);
}

```

```

int rw = rand() % NUM-RUNWAYS;
pthread-mutex-lock(&runway-lock[rw]);
if (!runway-busy[rw]) {
    runway-maint[rw] = !runway-maint[rw];
    printf("[MAINT] Runway %d maintenance %s\n", rw+1,  

        runway-maint[rw] ? "ON" : "OFF");
    if (!runway-maint[rw])
        pthread-cond-broadcast(&queue-cond);
}

```

```

    pthread-mutex-unlock(&runway-lock[rw]);
    pthread-mutex-unlock(&queue-lock);
}
return NULL;
}

```

```
int main()
```

```
    srand((unsigned) time(NULL));
```

```
    for (int i=0; i < NUM-RUNWAYS; i++)
```

```
        pthread_mutex_init(&runway-lock[i], NULL);
```

```
    memset(waiting-map, 0, sizeof(waiting-map));
```

```
    printf("Airport Control System\n")
```

```
    pthread_t gen, monitor; controllers[NUM-RUNWAYS];
```

```
    int rw-ids[NUM-RUNWAYS];
```

```
    pthread_create(&gen, NULL, flight-generator, NULL);
```

```
    pthread_create(&monitor, NULL, emergency-monitor, NULL);
```

```
    for (int i=0; i < NUM-RUNWAYS; i++) {
```

```
        rw-ids[i] = i;
```

```
        pthread_create(&controllers[i], NULL, runway-controller,
                        &rw-ids[i]);
```

```
    }
```

```
    pthread_join(gen, NULL);
```

```
    for (int i=0; i < NUM-RUNWAYS; i++) {
```

```
        pthread_join(controllers[i], NULL);
```

```
    pthread_mutex_lock(&queue-lock);
```

```
    pthread_cond_broadcast(&queue-cond);
```

```
    pthread_mutex_unlock(&queue-lock);
```



```
pthread_join (monitor, NULL);
```

```
//cleanup
```

```
pthread_mutex_destroy (& queue_lock);
```

```
pthread_cond_destroy (& queue_cond);
```

```
for (int i=0; i < NUM-RUNWAYS; i++) {
```

```
    pthread_mutex_destroy (& runway_lock[i]);  
}
```

```
printf ("All %d flights handled. \n", TOTAL-FLIGHTS);  
return 0;
```

```
}
```


⑤ Academic Review: Parallel Processing of E-Attheer Algorithm using p-thread paradigm

Introduction:

This paper addresses a well-known challenge in computer science: improving the speed of string matching algorithms when dealing with large datasets. As databases grow rapidly, sequential algorithms struggle to process data efficiently. The authors propose parallelizing the E-Attheer string matching algorithm using POSIX threads (Pthreads) to reduce execution time and improve overall performance across multiple database types.

Summary of the Paper:

The E-Attheer algorithm is an exact string matching algorithm that works in two stages (preprocessing and searching). During preprocessing, it builds shift tables using functions borrowed from the Boyer-Moore and Berry-Ravindran algorithms. During searching, it compares hash value of characters between the pattern and the text window, using these shift tables to skip unnecessary comparisons efficiently.

To parallelize this algorithm, the authors applied the Pthread paradigm. The text is divided into equal chunks, and each thread processes one chunk independently while sharing the pattern and shift tables.

Mutex locks are used to safely collect results from all threads at the end.

Experiments were conducted on a multi-core cluster using four database types (DNA, Protein, XML and Pitch) each with 200MB of data and both

Short and long pattern lengths.

Key Findings:

The parallel version consistently outperformed the sequential version in execution time. For short patterns, the best parallel results were 185ms, 96ms and 59ms using 2, 4, and 8 cores respectively, compared to a sequential best of 359ms. For speedup, the DNA database achieved the highest values (reaching up to $7.19\times$ with 8 cores on short patterns). The Pich database delivered the best parallel execution time overall. In contrast, XML and Protein databases showed the weakest speedup, particularly with long patterns, due to their larger alphabet sizes affecting the hash function's efficiency.

Strengths & limitations

The paper is well structured and its methodology is clearly explained. The use of real-world benchmark databases adds credibility to the results. The author test multiple variables (pattern length, core count, and database type) providing a thorough analysis. The flowchart in figure 1 effectively illustrate the parallel execution process, making it easier to understand the implementation logic.

However the paper has its limitations. The experiments are restricted to 8 cores, which limits conclusion about scalability. As the number of cores increases, communication overhead also grows, which the author acknowledge but didn't deeply analyzed. Additionally, the study only used one fixed data size (200 MB) so performance behavior with smaller or much larger data set is unknown.

Conclusion.

Overall the paper makes a solid contribution to the field of parallel computing by demonstrating that the E-Atter algorithms can be effectively parallelized using Pthreads with measurable performance gains. It is directly relevant to operating systems concepts such as thread management, synchronization and multi-core processing. The work is accessible, practically implemented and provides a useful foundation for future extensions using GPU or hybrid MPI-OpenMP models.